Reviewer Pack — Minimal Order of Quasi-monte Carlo Integration Points and Ci…

Entry e114a3904945 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 15:27:02 UTC

Minimal Order of Quasi-monte Carlo Integration Points and Circuit Monotone Width Inequality

Entry ID: `e114a3904945` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-06-04 15:27:02 UTC

1. Conjecture

Field A (mathematical branch): Quasi-monte Carlo Integration Theory **Field B** (complexity object): Boolean Circuit Complexity (Circuit Monotone Width)

Statement:

> For every n-vertex circuit C with monotone width w(C), the minimal number of points in a quasi-monte Carlo rule Q required to achieve an integral approximation error less than ε using the Euler-Maclaurin formula is O(w(C)^2 / log^2(1/ε)).

Rationale (proposer's reasoning):

> Quasi-monte Carlo integration provides better convergence rates compared to classical Monte Carlo methods. If this conjecture holds, it would establish a connection between the efficiency of quasi-monte Carlo methods and circuit complexity, potentially leading to faster algorithms for certain classes of problems.

Taxonomy category: `QUASI_MONTE_CARLO_CIRCUIT_COMPLEXITY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `a449c48cd46f456a`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> To support the conjecture, the integral approximation error must be less than ε using a quasi-monte Carlo rule with O(w(C)^2 / log^2(1/ε)) points, where w(C) is the monotone width of the circuit. To falsify the conjecture, for any given ε and w(C), the number of points Q exceeds O(w(C)^2 / log^2(1/ε)).

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	SAFE	1.00	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	SAFE	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - intitle:Quasi-monte Carlo Integration AND monotone width Boolean circuit complexity - arxiv:math.CA AND Euler-Maclaurin formula AND quasi-monte Carlo integration points - quant-ph AND monotone width AND circuit complexity AND quasi-monte Carlo rule

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def quasi_monte_carlo_rule(circuit, ε=1e-6):
        w_C = circuit['monotone_width']
        Q = int(Fraction(w_C**2, math.log(1/ε)**2))
        return Q

    def generate_monotone_circuit(n):
        # Placeholder for generating a random monotone circuit
        # This is a dummy implementation and should be replaced with actual logic
        circuit = {'monotone_width': n}
        return circuit

    def euler_maclaurin_integral(circuit, points):
        # Placeholder for calculating the integral using Euler-Maclaurin formula
        # This is a dummy implementation and should be replaced with actual logic
        return 0.0

    results = []
    for n in [5, 10, 15, 20, 30, 40]:
        circuit = generate_monotone_circuit(n)
        Q = quasi_monte_carlo_rule(circuit)
        error = euler_maclaurin_integral(circuit, Q)
        results.append({'n': n, 'Q': Q, 'error': error})

    metric_value = sum(result['error'] for result in results) / len(results)
    instances_tested = len(results)
```

```
    n_max = max(result['n'] for result in results)
    conjecture_holds = all(result['Q'] <= int(Fraction(result['circuit']['monotone_width']**2, mat

    return {
        "metric_name": "Integral Approximation Error",
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": "" if conjecture_holds else "mapping_undefined"
    }

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(result['metric_value'] for result in results) / len(results)
    support_fraction = sum(1 for result in results if result['conjecture_holds']) / len(results)

    if all(result['conjecture_holds'] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    elif any(not result['conjecture_holds'] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result['conj
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_fai
    else:
        print("RESULT: INCONCLUSIVE reason=mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fa37d34e.py", line 63, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fa37d34e.py", line 40, in run_trial
    Q = quasi_monte_carlo_rule(circuit)
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fa37d34e.py", line 23, in quasi_monte_c
    Q = int(Fraction(w_C**2, math.log(1/ε)**2))
            ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
   File "/usr/lib/python3.12/fractions.py", line 277, in __new__
    raise TypeError("both arguments should be "
TypeError: both arguments should be Rational instances
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed during execution, which prevents us from verifying whether the conjecture's support conditions were met. | next: Investigate the cause of the crash and attempt to run the test again with appropriate error handling to ensure it completes successfully.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	23850
2	preregistration	ollama_remote	glm4:latest	0	0	9820
3	novelty	ollama_remote	glm4:latest	0	0	8426
4	novelty	ollama_remote	glm4:latest	0	0	9140
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24153
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10000
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11433
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8413
9	judge	ollama_remote	glm4:latest	0	0	11984

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 117218 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/e114a3904945.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/e114a3904945.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/e114a3904945.tar.gz` (if generated)